Privileged Access Workstation Shenanigans – Michael Waterman

michaelwaterman.nl/2023/10/18/privileged-access-workstation-shenanigans

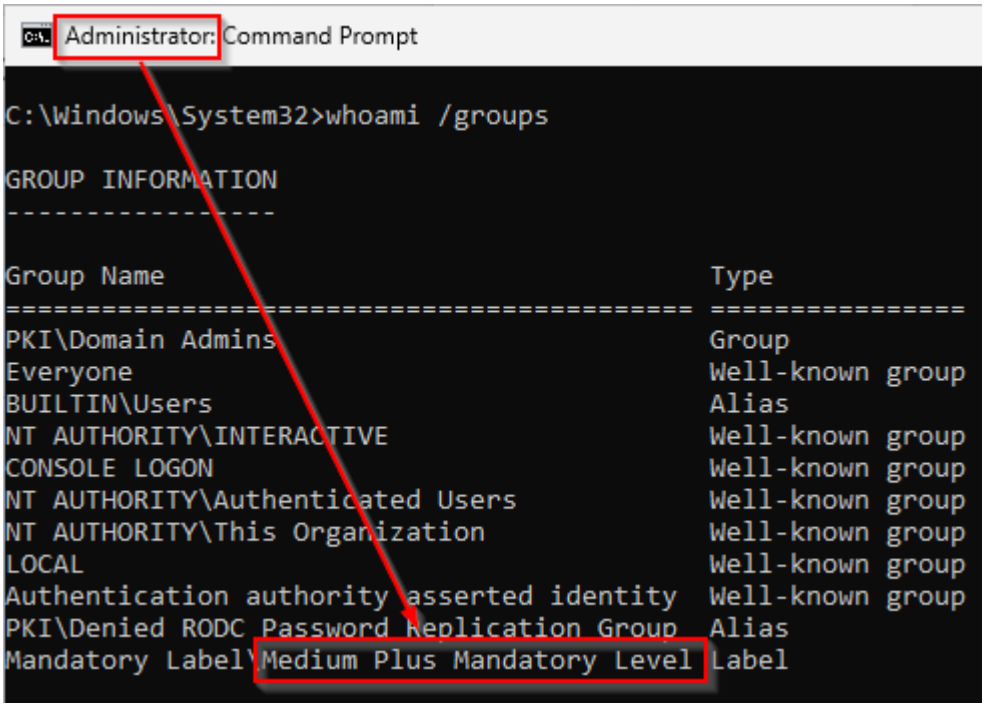
Michael Waterman October 18, 2023

Do you know what really ticks me off? Stuff that doesn't work as expected. Exactly that happened to me today while I revisited configuring a "*Privileged Access Workstation*" (PAW). Now a PAW is used to safeguard highly privileged credentials in a domain or cloud environment. In essence it's a workstation used solely for admin work, all infrastructure management is done from this machine. While talking about configuring a PAW is beyond the scope of this blog post, I do need to point out that "*Domain Admins*" and equivalent groups should never ever have local admin rights on a PAW, they should be regular users, reducing the risk of credential theft and the obvious malware infection that usually follows.

Anyways, during the configuration, I've used a group policy with restricted groups, removing the group "*Domain Admins*" from the local administrator group on all domain joined paw machines . After logging in again I tried to open one of the RSAT tools , "*Active Directory Users and Computers*", expecting it to just start. Instead I was greeted with a "*User Account Control*" prompt…… I definitely had a WTF moment, this shouldn't happen! Entering my password in the dialog started the application, but it was weird. I double checked my configuration and verified that the "*Domain Admins*" group was indeed removed from the local admins and unable to do any admin tasks. Just for the sake of argument I executed an elevated terminal and was greeted by the same UAC dialog. Entering my password actually also elevated the terminal and it said Administrator on the top left….. at that moment I thought I lost it. But there's more ways to check my sanity.

For those of you that understand how UAC actually works, the term "*Mandatory Integrity Controls*" (MIC) must ring a bell? Their often referred to as token levels. A standard user gets the MIC token of "*Medium Mandatory Level*" while admins get a token of "*High Mandatory Level*". While not a security boundary on it's self it does prevent processes in the same session to do just anything based on the highest level of rights that the logged in user has.

So I should be able to figure out what kind of token the user actually has. This can easily be discovered with the "*whoami*" tool adding the "*/groups*" parameter.



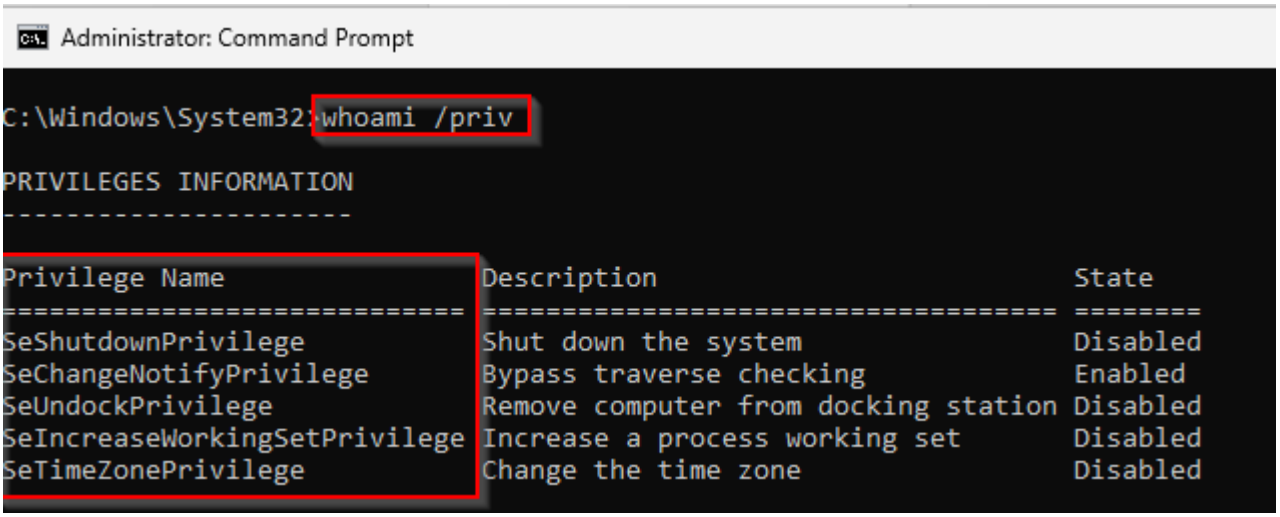
The results showed me "*Medium Plus Mandatory Level*", so that's a new one! I had to look that up. According to Microsoft this is the explanation for the "*plus*" part:

Applications that are launched with UIAccess rights for a standard user are assigned a slightly higher integrity level value in the access token. The access token integrity level for the UIAccess application for a standard user is the value of medium integrity level, plus an increment of 0x10. The higher integrity level for UIAccess applications prevents other processes on the same desktop at the medium integrity level from opening the UIAccess process object.

source: Windows Integrity Mechanism Design | Microsoft Learn

It turns out my user didn't have local admin, but was restricted to regular user plus a bit (yes, this is a word joke). Windows does this so other apps at the medium integrity level can just mess with our slightly elevated app.

Another way to determine what you can do on a system is check your privileges. Again, "whoami" to the rescue with "*whoami /priv*".

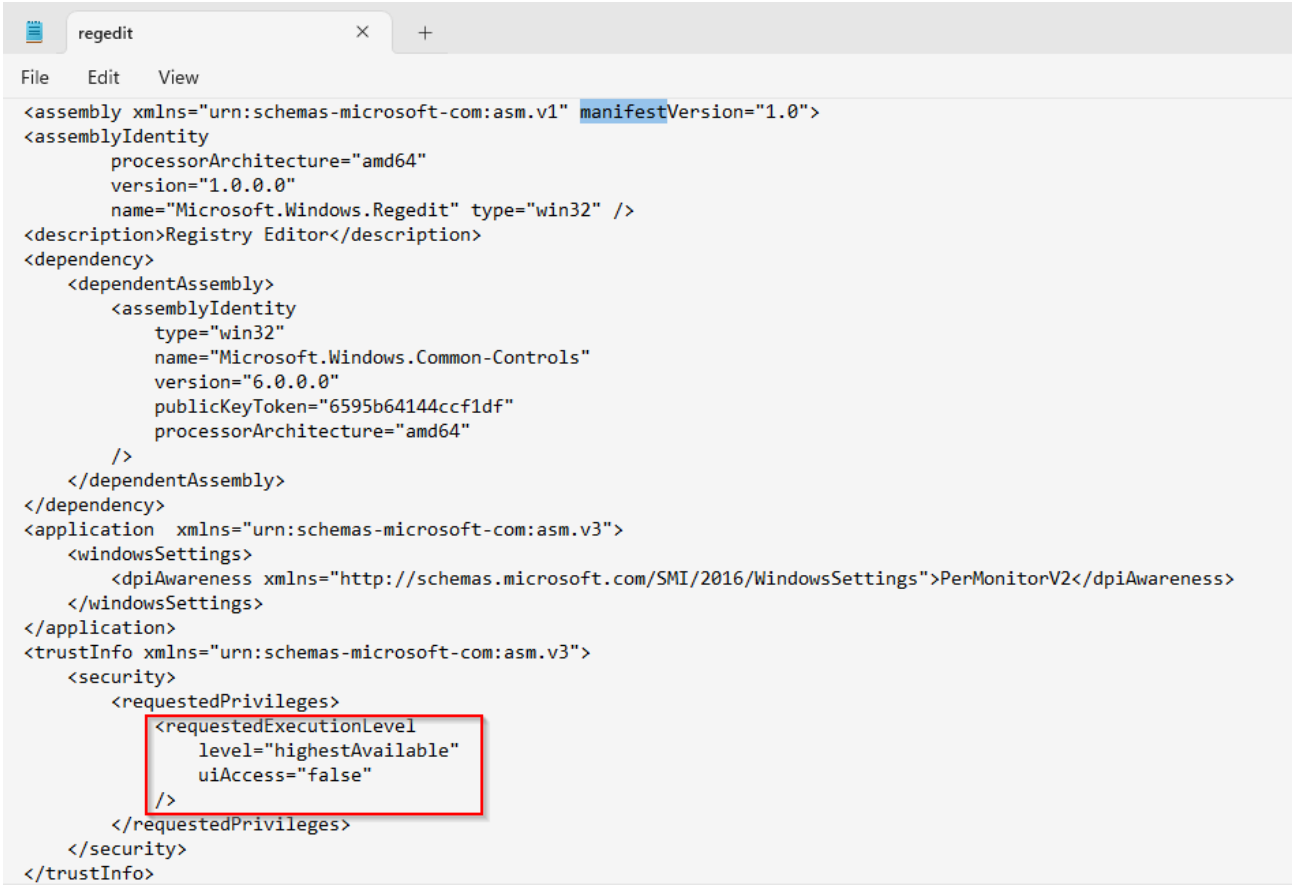


As you can see the list is very restricted, so my user is indeed no longer an admin, although there are indications, visual and behavioral that make it look like I am. The word "*Administrator*" in the console title and the invocation of UAC. Annoying at least to have to type in your password every time because of this. Especially when you're convincing your staff to use privileged access workstations and they need to type in their password 50 times a day….. can't win their hearts that way.

So what's the fix?

Using a manifest file

A what now? A manifest file is a definition file that's embedded into a binary file. It's actually very heavily used with the invocation of UAC. Every binary file created with Visual Studio 2005 (I believe) and up has a default manifest file. This file dictates the execution behavior of the binary file. Should I prompt for a UAC dialog, should I just use the available token etc. An easy inbox way of viewing the manifest file is dragging the binary into everyone's favorite text editor, Notepad.



Do a search for the word manifest and you'll see the configuration of the file, bet you didn't know that! Pay special attention to the "*requestedExecutionLevel*", this dictates the behavior of UAC. The options that can be used here are:

1. **asInvoker**: This level runs the application with the same privileges as its parent process. It does not request any special privileges and is the lowest privilege level. Use this if your application doesn't require elevated permissions.
2. **requireAdministrator**: This level requires the application to be run with administrator privileges. When the user launches the application, they will typically see a User Account Control (UAC) prompt asking for permission to elevate privileges.
3. **highestAvailable**: This level requests the highest privilege level available without requiring administrator rights. If the user is an administrator, the application will run with administrator privileges. If not, it will run with standard user privileges.
4. **uiAccess**: This level is used for applications that need to interact with the user interface of other applications, even when those applications are running with higher privilege levels. It's typically used for accessibility tools and requires special privileges and signing.

Now that we know this, what is the executionlevel of "*mmc.exe*", the framework that hosts the remote admin tools? Shouldn't it just use the "*Medium Mandatory Level*" token? Well yeah, it should, because the manifest of "*mmc.exe*" actually states "*highestAvailable*". But there's the catch, we don't have a "*Medium Mandatory Level*", we have a "*Medium **Plus** Mandatory Level*", that actually is responsible for the UAC prompt. It turns out that

privileged groups (like Domain Admins) always get a little extra regardless if you completely remove them from the local Administrator group, welcome to the mysterious world of the Windows platform, I still love it though!

Now that we know how a manifest file works, let's create one for ourselves. Simply copy the manifest content below and paste it in a new file that's next to mmc.exe, give it the name of "*mmc.exe.manifest*".

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">

<assemblyIdentity
    version="1.0.0.0"
    processorArchitecture="x86"
    name="Management Console"
    type="win32"
/>
<description>PAW Fix For Unwanted Elevation</description>

<trustInfo xmlns="urn:schemas-microsoft-com:asm.v3">
    <security>
        <requestedPrivileges>
            <requestedExecutionLevel
                level="asInvoker"
                uiAccess="false"
            />
        </requestedPrivileges>
    </security>
</trustInfo>
</assembly>
```

As you can see we've set the executionlevel to "*asinvoker*", basically telling Windows, use whatever token you have and take it from there.

But wait, we're not there just yet, there's one more step.

By default Windows doesn't search for external manifest files, that's more in the realm of application compatibility. So we first need to tell the operating system to look for additional config files. We can simply do that by adding a single registry key.

```
[HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\SideBySide]
"PreferExternalManifest"=dword:00000001
```

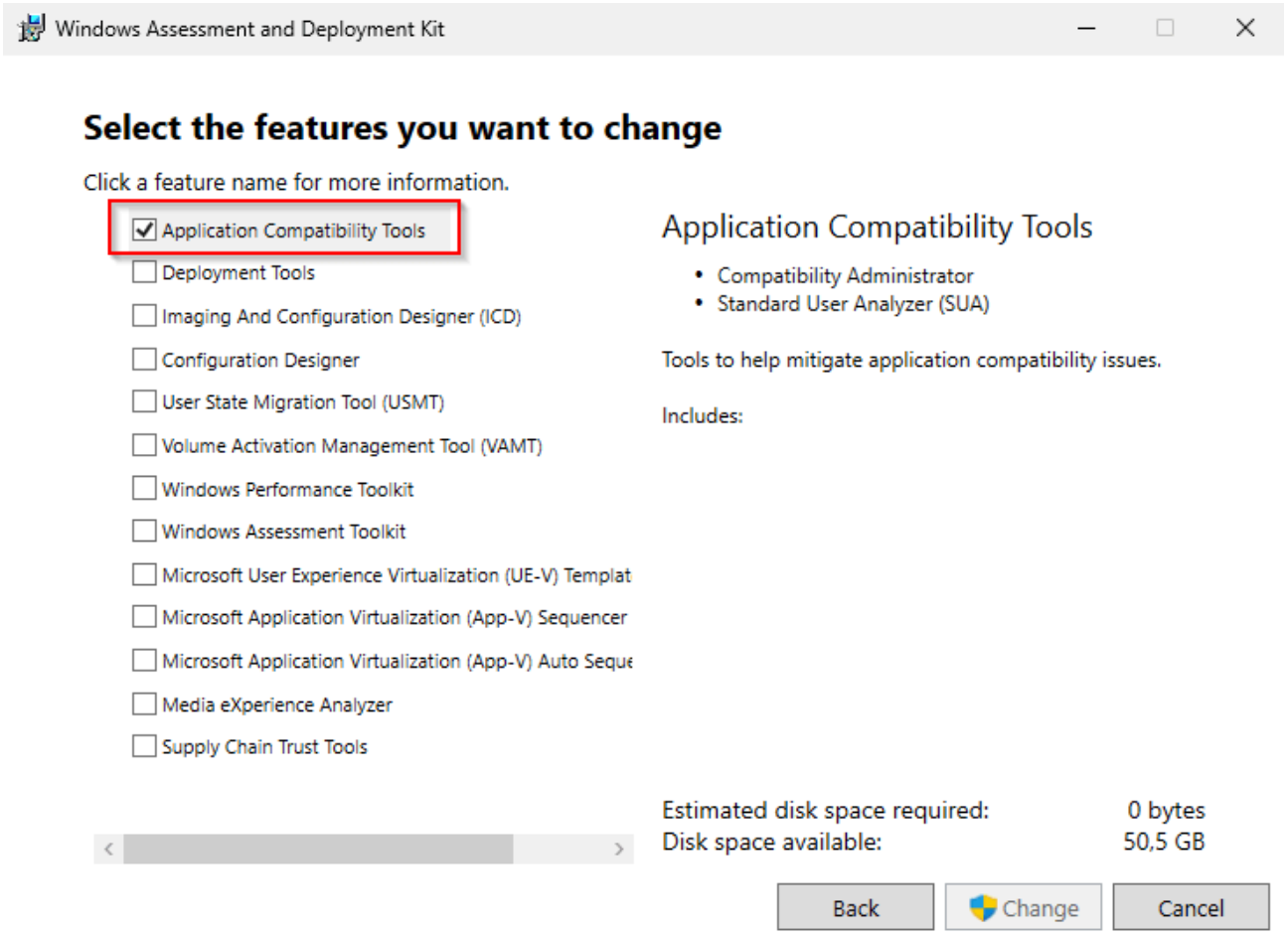
Reboot your machine and that should do it.

Application Compatibility Toolkit

If, for any reason you can't or are unwilling to set an entire system to accept external manifest files there's an easier and probably more secure fix that I thought of.

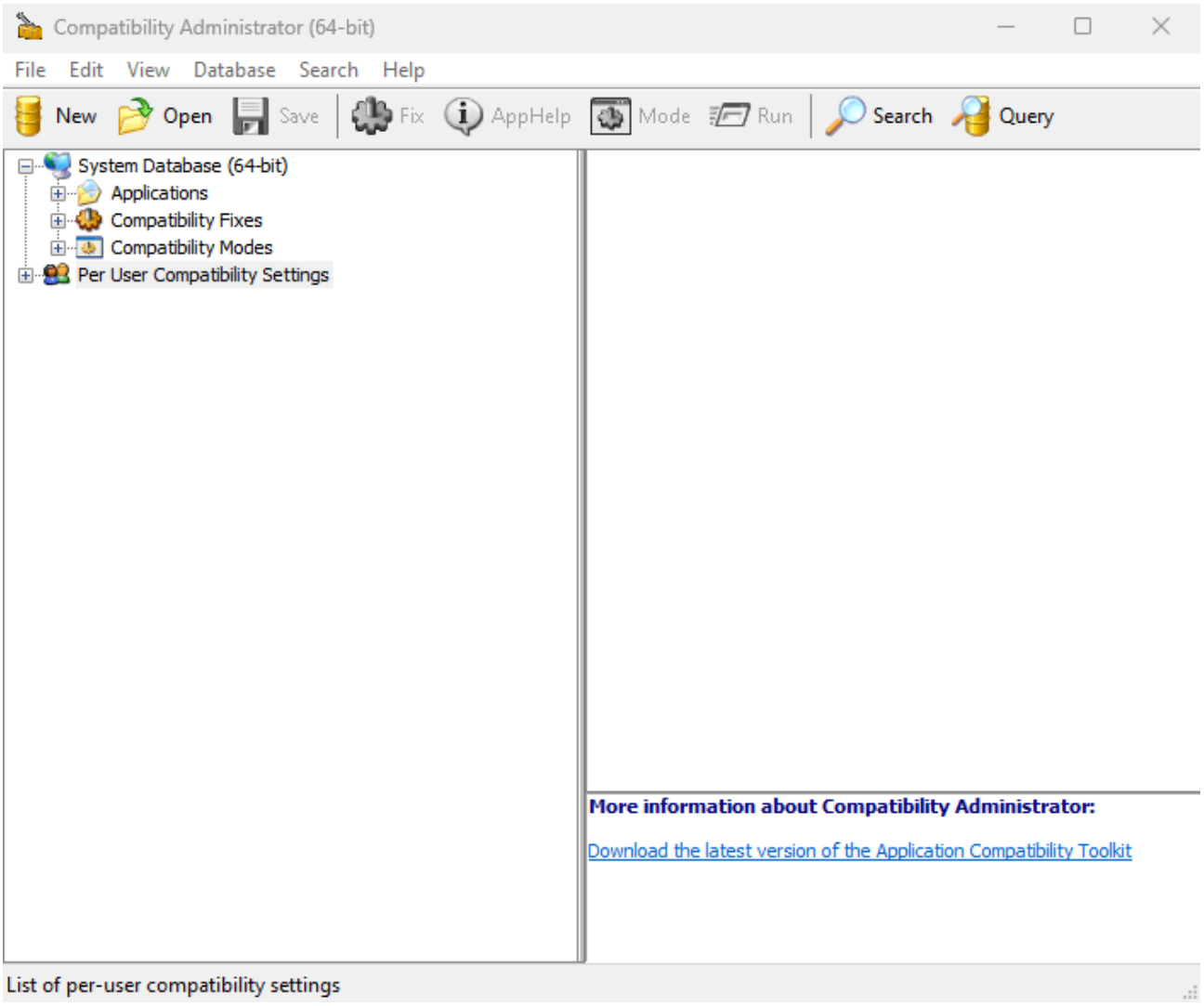
In a far distant past I was and end-point person while I still worked at Microsoft. When customers transitioned to a newer operating system version, application compatibility was sometimes an issue. I still remember the teachings of Chris Jackson, the AppCompat guy who unfortunately left us to soon. He always could get applications to play nice using the application compatibility toolkit, a part of the Windows Assessment and Deployment Kit. That's exactly what I will show you how to use the kit to "fix our issue".

First get the Windows Assessment and Deployment Kit from this link and install it. Only select the "A*pplication Compatibility Tools*" and finish the install.

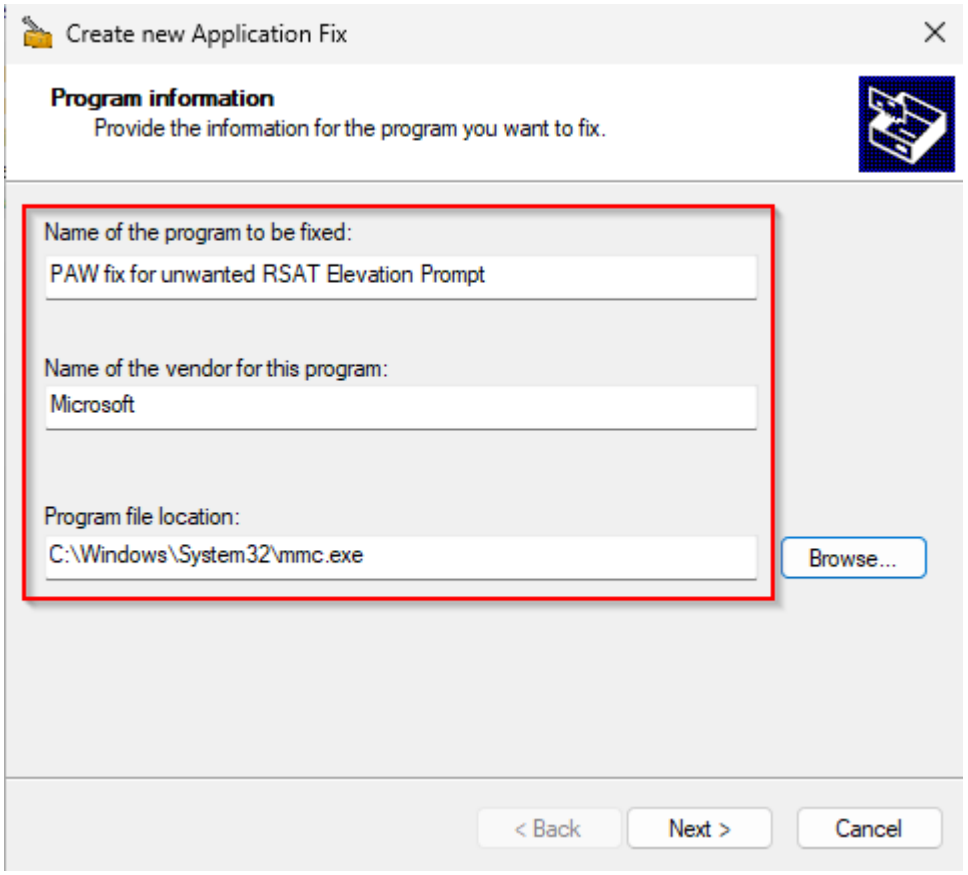


In your startmenu, search for the "*Compatibility Administrator (64-bit)*" and start it. Now comes the magic, I'll show you how to create a shim. To put it simply, a shim is a piece of code in between your application and the operating system. It's very well possible that you've used it before, but just didn't realize it. Suppose you need an application to behave like it's Windows 95. To do that, open the properties of the executable, select the "*Compatibility*" tab and select the operating system of choice. Under the hood, Windows will create and inject a shin with the "Version Lie" compatibility fix. Yes it's really named that way. Next, when the application starts and does a call to the OS asking it which version it is, the shim will tell the app it's Windows 95 instead of Windows 11. So many application could be fixed this way, but the application compatibility toolkit can do so much more than you can see in this dialog.

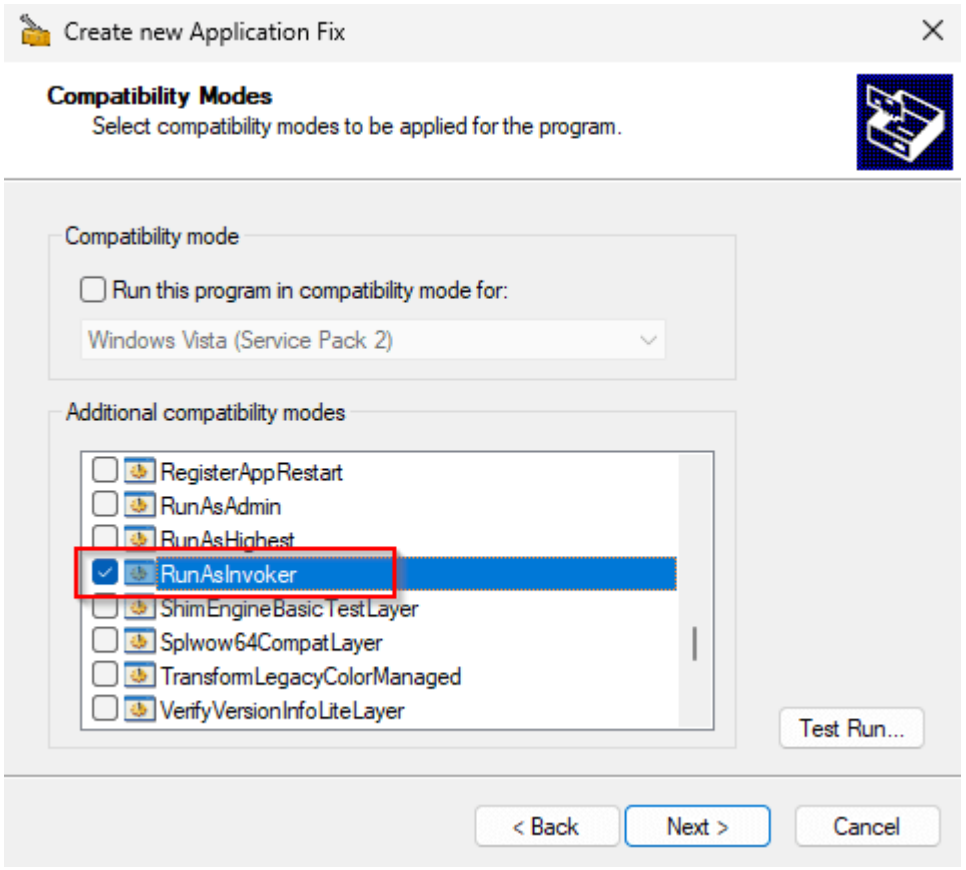
Back to the "*Compatibility Administrator*", I'll show you step by step on how to create a shim that fixes the unwanted elevation problem.



- On the top left, select "**New**". A new custom database will appear.
- Right click "**New Database**", "**Create New**", "**Application Fix"**.
- In the Program information dialog screen use the following:
  - **Name of the program to be fixed**: "PAW fix for unwanted RSAT Elevation Prompt".
  - **Name of the vendor for this program**: "Microsoft"
  - **Program File Location**: "C:\Windows\System32\mmc.exe"
- Click "**Next**".

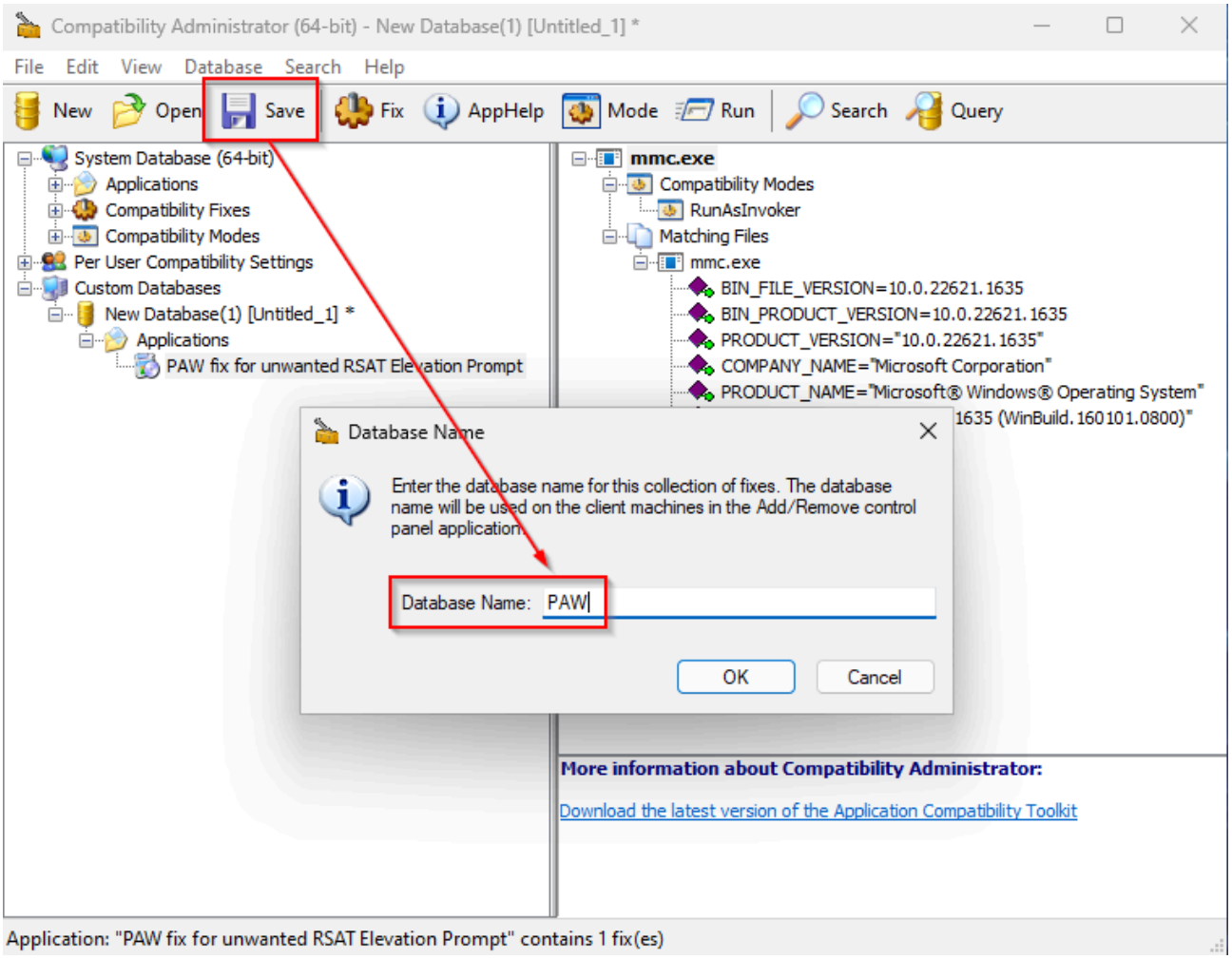


In the Compatibility modes dialog screen, scroll down in the list of fixes and select "**RunAsInvoker**", click "**Next**".



- On the Compatibility fixes dialog screen leave everything default and click "**Next**".

- On the Matching information dialog screen, leave everything default. This will detect the execution of the binary file by the parameters that are listed here. As you can see it takes a specific look at the file version. If you want to make it more generic, you could consider deselecting this and choose “ORIGINAL_FILENAME” instead, but that’s up to you. Click “**Finish**“.
- Click “**Save**” in the menu bar and name your database accordingly, I’ve named mine “PAW”, so I know what it’s used for. Click “**OK**“.
- In the Save Database: “PAW” dialog screen, select the location to store the database and give it a name. I usually give it the same filename as the database name.



Back in the “*Compatibility Administrator*“, the last step we need to take is register the shim. We can do that by right clicking on the newly created database and select “**Install**“. Click “**OK**” in the succeeding dialog screen.

And that’s it! If you need to distribute the shim to another machine you can use the command-line tool “**sdbinst.exe**” to manually register the shim. This tool is available on all Windows versions.

```
Usage: sdbinst [-?] [-q] [-u] [-g] [-p] [-n[:WIN32|WIN64]] myfile.sdb | {guid} |
"name"

    -? - print this help text.
```

```
-p - Allow SDBs containing patches.
-q - Quiet mode: prompts are auto-accepted.
-u - Uninstall.
-g {guid} - GUID of file (uninstall only).
-n "name" - Internal name of file (uninstall only).
```

Now that you know how this works, you're also able to create a new shim for the "Active Directory Administrative Center", which doesn't use the Microsoft Management Console. Happy shimming!

Download all the files I've used in this blog post here.

Hope this helps you with your adventures in using privileged access workstations and the adoption process. As always all feedback and remarks are welcome!